

SDK服务端接口文档

V1.3 配置查询

更新日期：2026-09-21

1. 接口功能

客户端在本轮用户登录成功后上报用户、设备、用户分类、首次打开时间、归因和首次广告收入信息。服务端重新计算用户分类，并一次返回广告配置和通知配置。

- 请求中的 `userType` 仅用于记录和对比，客户端以响应中的 `data.userType` 为准。
- 响应固定包含 `ad_config` 和 `notification_config` 两项，客户端通过每项的 `type` 识别配置。
- 某一项未匹配到配置时，不影响另一项正常返回。

2. 接口地址

项目	内容
Base URL	由部署环境提供，例如 https://api.example.com
请求方式	POST
接口路径	/server/user/searchConfigs
Content-Type	application/x-www-form-urlencoded; charset=UTF-8
响应格式	application/json; charset=UTF-8

完整地址： `{Base URL}/server/user/searchConfigs`

3. 请求头

Header	必传	说明	示例
appId	是	产品 App ID，与请求参数 appId 一致	20023
appVersion	是	当前客户端版本	1.0.0
sdkVersion	是	当前 SDK 版本	1.5.0
deviceId	是	稳定设备标识，与请求参数 deviceId 一致	android-device-uuid
platformId	是	发行平台 ID	Google Play Android 传 2
deviceTime	是	客户端当前 Unix 毫秒时间戳	1789458503000
zoneOffset	是	当前时区相对 UTC 的偏移小时数	中国时区传 8
Accept-Language	是	客户端当前语言	zh-CN、en-US

4. 请求参数

参数	类型	必填	说明
appId	integer	是	产品 ID
timestamp	string	是	Unix 秒级时间戳
sign	string	是	32 位大写 MD5 签名
uid	long	是	登录成功返回的用户 ID
deviceId	string	是	稳定设备标识
userType	string	是	客户端用户分类：RISK、NATURE、COMMON、HIGH_VALUE
firstOpenTime	string	是	首次打开时间，按设备本地时区传 yyyy-MM-dd HH:mm:ss
network	string	是	归因渠道；自然量传 Organic，未知传 UNKNOW
firstAdRevenue	string	是	首次广告收入；未知传 -1，已知零收入传 0
campaignID	string	否	广告系列 ID，未知可不传
campaignName	string	否	广告系列名称，未知可不传

表单示例：

代码块

```
1  appId=20023&timestamp=1789780000&uid=12345678&deviceId=android-device-
  uuid&userType=COMMON&firstOpenTime=2026-09-09
  14:56:12&network=google&firstAdRevenue=-1&campaignID=123&campaignName=launch&si
  gn={SIGN}
```

5. 验签

使用产品配置中的 `appKey` 生成签名：

1. 收集本次请求的全部非空表单参数，删除 `sign`。
2. 按参数名 ASCII 升序排列。
3. 拼接为 `key=value&key2=value2&`。
4. 末尾追加 `secretKey={appKey}`。
5. 对完整字符串计算 MD5，并转换为 32 位大写十六进制字符串。

上述表单示例对应的待签名原文：

代码块

```
1 appID=20023&campaignID=123&campaignName=launch&deviceID=android-device-  
  uuid&firstAdRevenue=-1&firstOpenTime=2026-09-09  
  14:56:12&network=google&timestamp=1789780000&uid=12345678&userType=COMMON&secre  
  tKey={APP_KEY}
```

注意：

- 所有已传递的非空业务参数都参与签名，包括 `uid`、`deviceID`、`userType`、`firstOpenTime`、`network`、`firstAdRevenue`、`campaignID` 和 `campaignName`。
- `firstOpenTime` 使用设备当前本地时区，不固定为北京时间，也不转换为 UTC；该字符串不携带时区偏移。
- 可选参数不传或传空时，不加入签名原文。
- `appKey` 只用于本地计算签名，不能作为请求参数上传。
- 验签失败返回 `code=-1`，客户端不应继续处理配置。

6. 返回参数

参数	类型	说明
code	integer	顶层 0 表示本轮查询完成；1060 表示未匹配到配置
msg	string	结果说明，客户端以 code 为准
serverTime	string	服务端 Unix 毫秒时间戳
data.userType	string	服务端分类：RISK、NATURE、COMMON、HIGH_VALUE
data.items	array	广告配置和通知配置两项结果
data.items[].type	string	ad_config 或 notification_config
data.items[].userType	string	本项使用的服务端用户分类
data.items[].code	integer	0 表示命中，1060 表示本项未匹配到配置
data.items[].id	string	配置 ID；未匹配时不返回
data.items[].versionNo	integer	配置版本号；未匹配时不返回
data.items[].config	object / null	完整配置 JSON；未匹配时为 null

成功响应示例：

代码块

```
1  {
2    "code": 0,
3    "msg": "operation success",
4    "serverTime": "1789780000000",
5    "data": {
6      "userType": "COMMON",
7      "items": [
8        {
9          "type": "ad_config",
10         "userType": "COMMON",
11         "code": 0,
12         "id": "101",
13         "versionNo": 3,
14         "config": {"example": "完整广告配置内容"}
15       },
16       {
17         "type": "notification_config",
18         "userType": "COMMON",
19         "code": 1060,
20         "config": null
21       }
22     ]
23   }
```

```
23     }  
24 }
```

客户端处理规则：

- 顶层 `code=0`：继续检查 `data.items`，每项 `code=0` 时应用服务端配置，`code=1060` 时该项使用客户端本地配置。
- 两项都未匹配时，顶层仍为 `code=0`，两项分别返回 `code=1060`。
- 顶层 `code=1060`：本轮不使用服务端配置，广告和通知均使用客户端本地配置。
- 验签失败、非 2xx 或响应格式异常：不使用本轮响应，继续使用客户端本地配置。
- 配置结构参考广告配置 `/raw/ad_config.json` 和通知配置 `/raw/notification_config.json`。

V1.2 用户登录接口

1. 接入范围

本期只接入：

1. SDK 初始化上报
2. 游客登录（首次调用时同时完成游客账号注册）
3. Token 登录和刷新（后续启动及已登录刷新时调用）
4. Singular 归因结果上报

暂不接入 Google、Facebook、Apple 等第三方登录。

客户端快速流程

1. 每次启动 App 初始化 SDK 后，都调用一次 `/server/user/initLog`；该接口用于记录本次设备信息。
2. 初始化 Singular 并持久化归因结果。
3. 没有有效 `uid + token` 时调用 `/server/user/platformLogin`；已有凭据时调用 `/server/user/autoLoginreflushtoken`。

4. 登录成功后保存用户信息，使用 `data.uid` 分别调用 `TDAnalytics.login(String.valueOf(uid))` 和 `Singular.setCustomUserId(String.valueOf(uid))`。
5. 登录成功后，如果本地已有 Singular 归因数据（媒体、广告系列、广告组和素材），调用 `/server/user/uploadUser`；失败则保留本地数据并重试。
6. 如果本次启动登录时尚未取得归因，等 Singular 回调取得后再调用一次 `/server/user/uploadUser`。
7. 后续每次启动登录成功后，都使用当前 `uid` 再调用一次 `/server/user/uploadUser`；接口按 `appID + uid` 幂等更新。

2. 基础约定

2.1 基础地址

当前线上 Server API 域名参考为：

代码块

```
1 https://api-server.xxx.com
```

实际域名由服务端按环境提供。

接口完整地址示例：`https://{SERVER_API_DOMAIN}/server/user/platformLogin`。

广告收入、广告展示和广告点击事件使用同一个 `SERVER_API_DOMAIN`，请求路径为 `/report/data/report`；客户端不需要单独配置 Report API 域名。详见[广告收入采集接口接入文档](./广告收入采集接口接入文档.md)。

2.2 请求方式和编码

- HTTP: POST
- Content-Type: application/x-www-form-urlencoded
- 字段名区分大小写，appID 中的 ID 必须保留大写
- 所有动态接口都必须携带 appID、timestamp、sign

2.3 请求头

本期新客户端调用以下四个接口时，都必须发送同一组公共请求头：

- /server/user/initLog
- /server/user/platformLogin
- /server/user/autoLoginreflushtoken
- /server/user/uploadUser

这些 Header 由客户端网络层统一添加，不是 DTO 表单字段，也不参与 sign 计算。后端通过 CheckLoginAspect 和 HttpUtils.getAppInfo() 读取并记录应用、版本、设备、平台、时间和语言信息。

Header	是否必传	字段意思	本期 Android 示例/取值来源
appID	是	产品 App ID，与请求体 appID 表示同一个产品	10019
appVersion	是	当前产品客户端版本	从 App 包版本读取，例如 1.0.0
sdkVersion	是	当前接入的 SDK版本	例如 1.5.0
deviceId	是	客户端稳定设备标识，用于设备关联和封禁校验	例如 android-device-uuid
platformId	是	发行平台 ID，不是 Android/iOS 标识	本次 Google Play Android 传 2

deviceTime	是	发起请求时的客户端设备时间，Unix 毫秒	例如 1787137000000
zoneOffset	是	设备当前时区相对 UTC 的偏移小时数	中国时区传 8
Accept-Language	是	客户端当前语言，用于服务端多语言消息和地区配置	例如 zh-CN 、 en-US

完整 Header 示例：

代码块

```
1 Content-Type: application/x-www-form-urlencoded
2 appID: 10019
3 appVersion: 1.0.0
4 sdkVersion: 1.5.0
5 deviceID: android-device-uuid
6 platformId: 2
7 deviceTime: 1787137000000
8 zoneOffset: 8
9 Accept-Language: zh-CN
```

请求头名称按以上写法统一发送。表单里的 appID 必须与 Header 中的 appID 保持一致；接口表单包含 deviceID 时，也必须与 Header 中的 deviceID 保持一致。platformId=2 表示 Google Play 平台；Android 操作系统标识由表单 channelId=0 表示，两者不能互换。

Android SDK 每次执行动态接口前自动补齐以下三个 **POST 表单字段**，不是上面的 Header：

表单字段	自动补齐规则	字段意思
appID	请求体没有该字段时，从 GlobalConfig.getAppID() 加入	产品 App ID；本次为 10019
timestamp		

	请求体没有该字段时，加入 <code>System.currentTimeMillis() / 1000</code>	发起请求时的 Unix 秒时间戳，注意不是毫秒
<code>sign</code>	请求体没有该字段时，使用当前全部非空表单参数和客户端密钥计算后加入	请求签名，用于后端验证请求参数没有被篡改

执行顺序是：业务代码先放入接口自己的参数，补齐 `appID` 和 `timestamp`，随后对全部非空表单参数计算 `sign`，最后发送请求。新客户端必须复用同一规则，不能遗漏这三个字段，也不能把 `sign` 写死。

Header `appID` 与表单 `appID` 都要发送：Header 用于服务端请求上下文和日志，表单字段用于 DTO、业务处理以及签名。Header 不参与签名；表单参数参与签名。

签名原文、排序、摘要算法应以现有 SDK `SignUtils.sign(params, clientKey)` 的实现为准；后端收到错误签名通常返回 `msg = "sign not matched"`。

签名规则

参与签名的是表单参数，不包含 Header：

1. 删除参数 `sign`，删除值为空的参数。
2. 按参数名 ASCII 升序排列。
3. 拼接为 `key=value&key2=value2&`。
4. 末尾追加 `secretKey={客户端密钥}`，再做 MD5，结果转大写。客户端密钥从运营后台“基本信息配置”中取得。

例如参数为 `accountID=guest-001`、`accountType=1`、`appID=10019`、`deviceID=device-001`、`timestamp=1787191000`、`channelID=0` 时，待签名字符串为：

代码块

```
1  accountID=guest-001&accountType=1&appID=10019&channelID=0&deviceID=device-001&timestamp=1787191000&secretKey={客户端密钥}
```

客户端密钥只保存在客户端安全配置或构建环境中，不要提交到公共仓库或写入本文档。

2.4 通用响应

代码块

```
1  {
2    "code": 0,
3    "msg": "operation success",
4    "serverTime": 1699930419179,
5    "data": {}
6  }
```

- `code == 0` : 成功
- `code != 0` : 业务失败，展示或记录 `msg`，不要只依据 HTTP 状态判断
- `serverTime` : 服务端毫秒时间，可用于校准客户端时间

登录成功的 `data` :

代码块

```
1  {
2    "uid": 123456789,
3    "name": "游客标识",
4    "loginName": "",
5    "displayType": 1,
6    "token": "服务端返回的 token",
7    "expiredTime": 2592000,
8    "registerTime": 1699930419179,
9    "countryCode": "US",
10   "accountType": 1,
11   "newAccount": 1
}
```

客户端至少持久化 `uid`、`token`、`name`、`accountType`、`registerTime`；
`expiredTime` 是秒数提示，当前服务端 token Redis TTL 为 30 天。

登录响应字段说明：

字段	类型	字段意思	客户端处理
<code>uid</code>	long	服务端用户唯一 ID	必须持久化，后续 Token 登录使用
<code>name</code>	string	当前账号展示/账号标识；游客通常是 <code>accountID</code>	持久化，作为 <code>username</code> 可回传
<code>loginName</code>	string	登录名；游客通常为 空	可展示或持久化
<code>displayType</code>	int	客户端展示用账号类型	按返回值展示
<code>token</code>	string	服务端登录凭据	必须持久化，禁止自行生成
<code>expiredTime</code>	long	Token 有效期提示，单位秒	仅作提示，不替代服务端校验
<code>registerTime</code>	long	注册时间，Unix 毫秒	可用于本地账号信息展示
<code>countryCode</code>	string/null	注册或识别出的国家/地区码	可选使用，允许为 空
<code>accountType</code>	int	账号类型；游客为 <code>1</code>	客户端保存并回传
<code>newAccount</code>	int	是否新账号： <code>1</code> 是， <code>0</code> 否	首次游客注册可用于埋点

2.5 登录成功后设置数数科技和 Singular 用户身份

用户通过 `/server/user/platformLogin` 或 `/server/user/autoLoginreflushtoken` 登录成功后，应使用响应中的 `data.uid` 同时设置数数科技和 Singular 的用户身份。数数科技会在之后采集的事件顶层加入 `#account_id`；Singular 会把该 ID 用于后续用户级数据、跨设备分析及已配置的数据导出。

代码块

```
1 // thinkingData 和 Singular SDK 均已初始化。
2 if (response.getCode() == 0 && response.getData() != null) {
3     long uid = response.getData().getUid();
4     saveUser(response.getData());
5     String userID = String.valueOf(uid);
6     thinkingData.login(userID);
7     Singular.setCustomUserId(userID);
8 }
```

调用时序和身份规则：

1. 必须在服务端认证成功、取得真实 `uid` 后设置两个平台的用户 ID，不要在请求登录接口前自行生成账号 ID。
2. 本项目没有角色维度时，使用服务端返回的 `uid`；如果产品明确按角色维度分析，则在创建角色或进入服务器后改用角色 ID。
3. 游客首次登录、已有游客再次登录、Token 登录和刷新成功后都执行同一身份同步；重复设置相同 `uid` 是允许的。
4. 用户切换账号时，先清除旧身份，再使用新账号的 `uid` 设置两个平台的用户 ID。
5. 用户主动退出账号时同时清除两个平台的用户身份，避免退出后采集的数据继续挂在旧账号下。

代码块

```
1 thinkingData.logout();
2 Singular.unsetCustomUserId();
```

Singular 提供两种设置方式，按取得 `uid` 的时机选择：

- 通常在 Singular SDK 初始化完成后，客户端才通过登录接口取得可信 `uid`，此时调用 `Singular.setCustomUserId(userID)`。
- 只有在初始化 Singular SDK 之前已经持有并确认可信 `uid` 时，才在配置中调用 `new SingularConfig(apiKey, secret).withCustomUserId(userID)`，让第一次 session 就携带用户 ID。
- `withCustomUserId` 是初始化配置方法，不能替代登录成功后的 `setCustomUserId`。本项目首次安装时初始化早于登录，因此必须保留登录成功后的调用。
- 设置 Singular 用户 ID 不会调用本项目的 `/server/user/uploadUser`，也不会把先前取得的匿名安装归因自动补写到业务库；客户端仍须执行第 3.5 节的归因同步。

不要把以下值设置为数数科技的 `#account_id`：

- 游客登录请求中的 `accountID`，它是客户端根据设备号生成的游客查找标识。
- `deviceID`、`#distinct_id` 或 `#uuid`，它们属于设备或匿名身份。
- 登录 Token，它是敏感凭据，不是分析身份。

广告事件必须在 `login` 设置完成后采集，批量上报时每条事件应能看到顶层 `#account_id`。详细事件格式参见 [广告收入采集接口接入文档](./广告收入采集接口接入文档.md)。

数数科技账号 ID 说明：

https://docs-v2.thinkingdata.cn/?version=v5.0&lan=zh-CN&code=user_identify&anchorId=>

数数科技 Android SDK 安装说明：

https://docs-v2.thinkingdata.cn/?version=v5.0&lan=zh-CN&code=android_sdk_installation&anchorId=>

重点参考该文档的 **3.1 设置账号 ID**，登录成功后调用

`TDAnalytics.login(String.valueOf(uid))` 设置用户身份，使后续事件自动携带顶层 `#account_id`。

Singular Android SDK 设置用户 ID：

<<https://support.singular.net/hc/zh-cn/articles/35636052267803-Android-SDK-%E8%AE%BE%E7%BD%AE%E7%94%A8%E6%88%B7-ID-%E5%92%8C%E5%93%88%E5%B8%8C-User-Details>>

2.6 游客 `accountID` 与设备号的生成规则

游客 `accountID` 不是直接使用设备号，也不是服务端返回的 `uid`。Android SDK 可以采用以下规则：

代码块

```
1  deviceID    = 本地持久化的设备标识
2  accountID   = MD5(deviceID)
```

客户端分别保存两个值：

值	客户端存储键	首次生成规则	用途
<code>deviceID</code>	<code>sf_device_id</code>	优先读取 Android <code>Settings.Secure.ANDROID_ID</code> ；为空时生成随机 UUID，再取 MD5	Header 和登录表单中的设备标识，参与设备关联/封禁校验
<code>accountID</code>	<code>sf_temp_uid</code>	对 <code>deviceID</code> 做 MD5： <code>MD5(deviceID)</code>	游客登录表单的账号标识，服务端按它找回同一个游客账号

首次安装的 Android 游客登录步骤应是：

1. 读取 `sf_device_id`；没有则生成并保存 `deviceID`。
2. 读取 `sf_temp_uid`；没有则计算 `MD5(deviceID)` 并保存为 `accountID`。

3. 使用这个 `accountID` 调用 `/server/user/platformLogin`，并在后续请求中持续复用。

因此，新客户端不能把 `accountID` 直接写成 Android ID，也不能每次启动随机生成。卸载重装会清除 SDK 保存的 `sf_temp_uid`；如果重新读取到的 `ANDROID_ID` 与原来相同，再计算 `MD5(deviceID)` 通常仍能找回原游客；如果设备号也发生变化或无法恢复，计算出的 `accountID` 就会变化，服务端会把它视为新的游客账号。这不是服务端 Token 丢失，而是客户端账号标识发生了变化。

PowerShell 签名和请求示例（密钥从环境变量读取，不要写进代码仓库）：

代码块

```
1  $secret = $env:SF_CLIENT_KEY
2  $p = [ordered]@{
3      accountID='a7625b06de72fe7c40da27cbc123c0c7'; accountType='1'; appID='10019'
4      channelID='0'; deviceID='device-001'; timestamp=
5      [DateTimeOffset]::UtcNow.ToUnixTimeSeconds()
6  }
7  $raw = ((($p.Keys | Sort-Object | ForEach-Object { "$_=${$p[$_]}" }) -join ' ' )
8  + "secretKey=$secret"
9  $md5 = [System.Security.Cryptography.MD5]::Create()
10 $sign =
11 ([System.BitConverter]::ToString($md5.ComputeHash([Text.Encoding]::UTF8.GetBytes($raw))).Replace('-', '')).ToUpperInvariant()
12 $body = $p + @{ sign=$sign }
13 Invoke-RestMethod 'https://api-server.xxx.com/server/user/platformLogin' -
14 Method Post -ContentType 'application/x-www-form-urlencoded' -Headers @{
15     appID='10019'; appVersion='1.0.0'; sdkVersion='1.5.0'; deviceID='device-
16     001'; platformId='2'
17 } -Body $body
```

该示例会真实创建或登录游客账号，请只使用测试设备标识；生产调用前先确认 `SF_CLIENT_KEY` 和 `appID` 环境配置。

2.7 Android 平台、账号和渠道字段取值

这三个字段含义不同，不能互相替代：

字段	所在位置	表示什么	当前 Android 游客接入值
platformId	请求 Header	发行平台/商店渠道	2 （Google Play）
accountType	登录请求表单、登录响应	用户账号类型	1 （游客）
channelID	/server/user/initLog、 /server/user/platformLogin 表单	客户端操作系统平台	0 （Android）

platformId：发行平台取值

后端 Consts.Platform 当前定义如下。客户端发送到 Header platformId，不是表单字段；后端会用它参与互通、渠道和平台配置判断。

值	后端常量	含义	Android 客户端何时使用
0	未指定/默认	未指定平台；部分支付逻辑按 Google Play 默认处理	不建议主动使用，除非后台明确要求
1	AppStore	Apple App Store	Android 不使用
2	GooglePlay	Google Play	当前 Google Play Android 包使用

accountType：账号类型取值

值	含义	本期是否使用
1	游客	是
2	邮箱	否

3	Google	否
4	Facebook	否
5	Apple	否

以上取值对应后端 `Consts.AccountType` 。本期 Android 只传 `accountType=1` ；游客登录时， `accountID` 是客户端生成并持久化的游客标识。

`channelID` ：客户端平台取值

值	含义	当前使用建议
0	Android	当前 Android 客户端必须传 0 ；
1	iOS	iOS 客户端传 1 ；本期 Android 不使用

客户端必须在初始化和游客登录中保持同一个 `channelID` 。它与 Header `platformId` 是两个不同字段： `channelID` 表示 Android/iOS， `platformId` 表示发行平台/商店（Google Play、AppStore 等）。不要用 `platformId` 替代 `channelID` 。

当前 Android 游客登录固定示例：

代码块

```
1 Header: platformId=2
2 Form:   accountType=1&channelID=0
```

不要发送 `platformId=1` 来表示 Android，也不要把 `channelID` 写成 2 来表示 Google Play；前者是 App Store，后者不是合法的当前平台值。

3. 推荐接入时序

3.1 Android 客户端的真实调用结论

客户端实际运行时，后续启动的 Token 登录和已登录状态刷新使用同一个接口。

接口	客户端触发时机	新客户端接入建议
<code>/server/user/autoLogin</code> <code>reflushtoken</code>	用户点击/触发 SDK 登录时，本地存在仍在有效期内的 <code>uid + token</code> ；以及登录状态下主动刷新用户、断网恢复后补刷	作为后续打开 App 时的 Token 登录接口，同时用于已登录状态刷新凭据
<code>/server/user/platformLogin</code>	本地没有有效用户，或 Token 登录和刷新接口发生重试的业务失败后	使用持久化的游客 <code>accountID</code> 登录；不存在时服务端自动注册

这里的“本地凭据有效”由客户端 `SFLoginHelper.isUserValid()` 判断：本地用户不为空、`uid` 和 `token` 不为空，并且距离上次保存登录信息的时间尚未超过 `expiredTime - 60` 秒。它只是决定客户端走哪个分支，最终仍以服务端响应为准。

3.2 首次安装并打开 App

代码块

```
1  flowchart TD
2      A[打开 App] --> B[初始化 SDK]
3      B --> C[POST /server/user/initLog]
4      C --> D[产品调用 SDK login]
5      D --> E[读取本地登录用户]
6      E --> F{存在有效 uid 和 token?}
7      F -- 否, 首次安装 --> G[读取或生成 deviceID]
8      G --> G2[accountID = MD5(deviceID), 持久化保存]
9      G2 --> H[POST /server/user/platformLogin<br/>accountType=1, channelId=0]
10     H --> I{code == 0?}
```

```
11 I -- 是 --> J[保存 accountID、uid、token、用户信息和登录时间]
12 J --> J2[设置数数和 Singular 的用户 ID<br/>并上报已持久化的 Singular 归因]
13 J2 --> K[通知产品登录成功并进入产品]
14 S[若归因回调晚到，取得后再调用 /server/user/uploadUser] --> J2
15 I -- 否 --> L[通知登录失败，不进入产品]
```

首次安装没有本地 `uid/token`，所以不会调用 Token 登录和刷新接口。`/platformLogin` 第一次通常返回 `newAccount=1`，同时完成游客注册和登录。

3.3 后续再次打开 App

代码块

```
1 flowchart TD
2   A[再次打开 App] --> B[初始化 SDK]
3   B --> C[POST /server/user/initLog]
4   C --> D[产品调用 SDK login]
5   D --> E[读取本地 uid、token、expiredTime 和 loginTime]
6   E --> F{本地凭据仍在有效期内?}
7   F -- 是 --> G[先恢复本地登录状态]
8   G --> H[POST /server/user/autoLoginreflushtoken]
9   H --> I{code == 0?}
10  I -- 是 --> J[覆盖保存服务端返回的用户信息和 token]
11  J --> J2[设置数数和 Singular 的用户 ID<br/>并上报已持久化的 Singular 归因]
12  J2 --> K[进入产品]
13  S[若归因回调晚到，取得后再调用 /server/user/uploadUser] --> J2
14  I -- 否，可重试业务错误 --> L[删除失效的本地登录用户]
15  L --> M[使用原游客 accountID<br/>POST /server/user/platformLogin]
16  M --> N{code == 0?}
17  N -- 是 --> J
18  N -- 否 --> O[退出登录并通知失败]
19  I -- 否，封禁或退款封禁 --> O
20  F -- 否或本地字段缺失 --> M
```

游客刷新失败后重新调用 `/platformLogin` 时，必须继续使用原来持久化的游客 `accountID`。不要生成新的 `accountID`，否则服务端会当成另一个游客账号，原产品进度无法关联。

3.4 断网与补偿逻辑

- 启动时没有网络且没有本地有效用户：客户端记录“等待自动登录”；网络恢复后重新执行登录判断。
- 启动时有本地有效用户但没有网络：客户端先把缓存用户恢复为当前登录态，并记录“等待刷新Token”；网络恢复后调用 `/server/user/autoLoginreflushtoken` 补刷。
- Token 登录和刷新接口返回 `code <= 0` 的网络/系统错误：客户端保留本地用户和登录记录，等待后续重试。
- Token 登录和刷新接口返回可重试的正数业务码：客户端删除失效登录用户，游客账号使用原 `accountID` 回退到 `/server/user/platformLogin`。
- 用户封禁、退款封禁等不可重试业务码：直接退出登录，不自动游客重登。

`/server/user/autoLoginreflushtoken` 同时承担 Token 登录和刷新职责，但它不是使用 refresh token 换取 access token 的标准刷新接口；它要求现有 token 能通过后端校验。

3.5 Singular 归因上报时序

Singular 的归因回调和用户登录是两个异步过程。客户端不能假设归因一定先于登录返回，应分别持久化登录状态和归因结果，并在两者都具备后再上报：

1. App 初始化 Singular SDK，通过 `SingularConfig.withSingularDeviceAttribution` 注册回调并获取归因结果。
2. 回调到达后，立即把实际返回的媒体、广告系列、广告组和素材字段持久化到本地；此时用户可能尚未登录。
3. 游客注册、游客登录或 Token 登录成功并取得服务端 `uid` 后，先调用 `Singular.setCustomUserId(String.valueOf(uid))`，再调用 `/server/user/uploadUser` 绑定本地归因和业务用户。
4. 只有接口返回 `code == 0` 才视为本次同步成功。网络错误或 `code != 0` 时保留本地归因结果，使用有限次数重试和退避策略，避免高频请求。
5. 后续每次 App 启动并登录成功后，使用当前 `uid` 再调用一次 `/server/user/uploadUser`。接口按 `appID + uid` 幂等更新，重复上报不会新增多条同来源归因记录。

Singular 回调字段和本项目接口字段的映射如下：

Singular 回调字段	本地/接口字段	含义
<code>network</code>	<code>network</code>	归因媒体网络；自然量按回调实际值保存
<code>campaign_id</code>	<code>campaignID</code>	广告系列 ID
<code>campaign_name</code>	<code>campaignName</code>	广告系列名称
<code>subcampaign_id</code>	<code>adGroupID</code>	广告组/子广告系列 ID
<code>subcampaign_name</code>	<code>adGroupName</code>	广告组/子广告系列名称
<code>creative_id</code>	<code>creativeID</code>	素材 ID
<code>creative_name</code>	<code>creativeName</code>	素材名称

除 `network` 外，其余字段仅在归因触点提供时才会出现；`subcampaign_*` 和 `creative_*` 并非所有媒体都提供。客户端必须先判断 key 是否存在且值非空，缺失时不传对应接口参数，不要用空字符串或猜测值占位。自然量应使用 Singular 回调实际返回的网络值。

代码块

```
1 SingularConfig config = new SingularConfig(apiKey, secret)
2   .withSingularDeviceAttribution(attributionData -> {
3       saveAttribution(
4           valueOf(attributionData, "network"),
5           valueOf(attributionData, "campaign_id"),
6           valueOf(attributionData, "campaign_name"),
7           valueOf(attributionData, "subcampaign_id"),
8           valueOf(attributionData, "subcampaign_name"),
9           valueOf(attributionData, "creative_id"),
10          valueOf(attributionData, "creative_name")
11       );
12   });
```

官方字段说明（`withSingularDeviceAttribution`）：

<<https://support.singular.net/hc/zh-cn/articles/37157782717851-Android-SDK-%E9%85%8D%E7%BD%AE%E6%96%B9%E6%B3%95%E5%8F%82%E8%80%83>>

本项目不依赖 Singular Internal BI Postback 完成用户归因入库；客户端回调加 `/server/user/uploadUser` 是业务库的归因同步链路。

4. 接口明细

4.1 SDK 初始化上报

`POST /server/user/initLog`

必填字段：

字段	类型	说明
<code>appID</code>	int	产品 App ID
<code>timestamp</code>	string	Unix 秒时间戳
<code>sign</code>	string	按客户端密钥计算
<code>deviceID</code>	string	客户端稳定设备标识
<code>channelID</code>	int	客户端操作系统平台； Android 固定传 <code>0</code> ，iOS 传 <code>1</code>

可选字段及取值说明：

字段	类型	说明

deviceType	string	设备型号，例如手机型号
manufacturer	string	设备制造商，例如 <code>samsung</code> 、 <code>Xiaomi</code> ；非必传
ram	string	用户设备的总内存，单位 GB，例如 <code>16</code> ；只传数值，不包含单位；非必传
disk	string	用户设备的总存储，单位 GB，例如 <code>256</code> ；只传数值，不包含单位；非必传
deviceOS	int	设备系统： <code>1</code> Android， <code>2</code> iOS；Android 传 <code>1</code>
deviceDpi	string	设备屏幕的物理像素密度 DPI
mac	string	设备 MAC 地址；无法获取时可不传或传空值

Android 初始化建议传 `deviceOS=1`、`channelID=0`。

客户端实际请求体示例（`sign` 需按上面的规则计算）：

代码块

```
1 appID=10019&timestamp=1787191000&sign={SIGN}&deviceId=device-001&channelID=0&mac=&deviceType=Pixel_8&deviceDpi=420&deviceOS=1
```

成功只表示设备记录上报成功：

代码块

```
1 {"code":0,"msg":"operation success","serverTime":1699930419179}
```

4.2 游客登录 / 首次游客注册

POST /server/user/platformLogin

请求字段：

字段	必填	示例	说明
appID	是	10019	产品 App ID
timestamp	是	1699930419	Unix 秒
sign	是	...	签名
accountType	是	1	必须为游客
accountID	是	MD5(deviceID)	游客账号标识；
deviceID	是	android-id-or-generated-id	稳定设备 ID
channelID	是	0	客户端操作系统平台；本期 Android 固定传 0，并参与签名
loginName	否	空	游客可不传
extraData	否	空	游客可不传

服务端按 appID + accountType + accountID 查找账号；不存在时创建游客账号，因此该接口同时覆盖游客注册和游客登录。服务端还会按 deviceID 和产品配置执行游客次数限制。

客户端实际请求体示例：

代码块

```
1 appID=10019&timestamp=1787191000&sign={SIGN}&accountType=1&accountID=a7625b06de72fe7c40da27cbc123c0c7&deviceID=device
```

-001&channelID=0

其中 `accountID` 必须持久化。卸载重装后如果该值变化，服务端会认为是新游客。

成功: `data.newAccount = 1` 表示本次新建，`0` 表示已有游客账号。

成功响应示例（字段值为示例，真实 `uid/token/registerTime` 由服务端生成）：

代码块

```
1  {
2    "code": 0,
3    "msg": "operation success",
4    "serverTime": 1787137000123,
5    "data": {
6      "uid": 987654321,
7      "name": "guest-device-001",
8      "loginName": "",
9      "displayType": 1,
10     "token": "generated-by-server",
11     "expiredTime": 2592000,
12     "registerTime": 1787137000120,
13     "countryCode": "US",
14     "accountType": 1,
15     "newAccount": 1
16   }
17 }
```

常见失败码：

code	含义
1001	产品不存在
1004	游客注册被关闭

1005	游客注册次数超过配置
1006	平台登录失败
1003	用户/IP/设备被封禁
1022	当前地区不可用

4.3 Token 登录和刷新

POST /server/user/autoLoginreflushtoken

该接口承担两个用途：后续打开 App 时，使用本地保存的 uid + token 完成 Token 登录并恢复登录状态；用户已经登录时，重新校验 Token 并刷新服务端返回的用户信息和登录凭据。两个场景使用相同的请求参数和后端实现。

请求字段：appID、timestamp、sign、uid、token、deviceID；建议同时传 accountType 和 username。

后端 autoLoginreflushtoken() 直接调用统一的 tokenLogin() 实现，负责产品校验、地区/IP 校验、uid/token/设备校验、封禁校验和登录结果返回。

客户端在以下时机调用本接口：

- 1. SDK 的 login() 被调用，且本地 uid/token 通过有效期预检查。
- 2. 产品在已登录状态主动调用 SDK 的 refreshUser()。
- 3. 启动刷新时网络不可用，之后监听到网络恢复并执行补刷。

它不是固定定时任务，也不需要先调用其他 Token 接口。启动恢复登录时，客户端直接调用本接口。

注意：该接口不是传统意义上使用 refresh token 换取新 access token 的接口。旧 token 无效时，该接口也会返回登录失败。成功后仍应保存响应中的 token；当前后端可能返回与旧 token 相同的 token，客户端不要假设每次都会变化。

客户端实际请求体示例：

代码块

```
1 appID=10019&timestamp=1787191070&sign={SIGN}&uid=1375560669270130159&token={PLATFORM_LOGIN返回的TOKEN}&deviceID=device-001&accountType=1&username=codex-test-10019-1787191469573
```

无效签名响应为：

代码块

```
1 {"code":-1,"msg":"sign not matched","serverTime":1787137675246}
```

`serverTime` 会随请求变化。使用正确签名后，才会进入产品不存在、token 无效、游客限制等业务分支。

4.4 Singular 归因结果上报

`POST /server/user/uploadUser`

接口使用 `application/x-www-form-urlencoded`，必须携带第 2.3 节的公共 Header，并按相同规则对全部非空表单参数签名。Header 中的 `deviceID` 用于同步设备归因信息，不作为该接口的表单字段参与签名。

字段	必填	类型	说明
<code>appID</code>	是	int	产品 App ID
<code>timestamp</code>	是	string	请求发起时的 Unix 秒时间戳

sign	是	string	使用客户端密钥计算的 32 位大写 MD5
uid	是	long	登录成功响应中的 data.uid，不要传游 客 accountID
network	是	string	Singular 返回的归因媒体 网络；自然量也使用 Singular 的实际返回值
campaignID	否	string	Singular 返回的广告系列 ID；自然量或无系列信息 时可不传
campaignName	否	string	Singular 返回的广告系列 名称；自然量或无系列信 息时可不传
adGroupID	否	string	Singular subcampaign_id，广 告组/子广告系列 ID；回 调未提供时不传
adGroupName	否	string	Singular subcampaign_name， 广告组/子广告系列名 称；回调未提供时不传
creativeID	否	string	Singular creative_id，素材 ID；回调未提供时不传
creativeName	否	string	Singular creative_name，素材 名称；回调未提供时不传

示例表单参数：

代码块

```
1 appID=10019&uid=1375560669270130159&network=Google  
  Ads&campaignID=123456789&campaignName=US_AOS_group1&adGroupID=987654321&adGroup
```

Name=US_AOS_adset1&creativeID=456789123&creativeName=video_01×tamp=1787191100&sign={SIGN}

签名时参数仍按 ASCII 升序排列。上述非空参数对应的签名原文为：

代码块

```
1 adGroupID=987654321&adGroupName=US_AOS_adset1&appID=10019&campaignID=123456789&campaignName=US_AOS_group1&creativeID=456789123&creativeName=video_01&network=Google Ads&timestamp=1787191100&uid=1375560669270130159&secretKey={客户端密钥}
```

成功响应：

代码块

```
1 {"code":0,"msg":"operation success","serverTime":1787191100123}
```

客户端应在登录成功并确认 `appID + uid` 正确后上报，后续启动可继续按幂等规则重复同步。

5. 线上实测记录（appID=10019）

测试时间：2026-08-20。测试环境及参数：

项目	值
----	---

API 域名	<code>https://api-server.xxx.com</code>
App ID	<code>10019</code>
Android 包名	<code>com.topgame.smaple</code>
<code>platformId</code>	<code>2</code>
<code>accountType</code>	<code>1</code> （游客）
客户端密钥	已用于签名，文档中脱敏
token	脱敏；由游客登录响应取得，禁止硬编码

5.1 初始化

`POST /server/user/initLog` 返回 HTTP `200`：

代码块

```
1  {"code":0,"msg":"operation success","serverTime":"1787191499937"}
```

5.2 首次游客注册/登录

使用全新的 `accountID` 和 `deviceID` 调用 `POST /server/user/platformLogin`，返回 HTTP `200`：

代码块

```
1  {
2    "code": 0,
3    "data": {
4      "accountType": 1,
5      "countryCode": "HK",
6      "displayType": 1,
```

```
7     "expiredTime": "2592000",
8     "loginName": "Visitor",
9     "name": "codex-test-10019-1787191469573",
10    "newAccount": 1,
11    "registerTime": "1787191500773",
12    "token": "[REDACTED_TEST_TOKEN]",
13    "uid": "1375560669270130159"
14  },
15  "serverTime": "1787191500721"
16 }
```

实测说明：首次注册 `newAccount=1`；当前 `platformLogin` 成功响应没有输出 `msg`，客户端必须以 `code==0` 判断成功，不能要求 `msg` 必须存在。

5.3 Token 登录和刷新（客户端后续启动实际接口）

继续调用 `POST /server/user/autoLoginreflushtoken`，返回 HTTP `200`、`code=0`、`newAccount=0`；本次测试返回的 token 与游客登录 token 相同。客户端不要假设该接口每次都会颁发不同的新 token。

5.5 异常响应

错误签名（例如直接传 `sign=INVALID_SIGN`）仍返回 HTTP `200`，业务码为 `-1`：

代码块

```
1  {"code":-1,"msg":"sign not matched","serverTime":1787191503601}
```

签名正确但 token 错误时，`POST /server/user/autoLoginreflushtoken` 返回 HTTP `200`，业务码为 `1002`：

代码块

```
1 {"code":1002,"msg":"login failed","serverTime":"1787191504276"}
```

测试账号本次实测 `uid` 为 `1375560669270130159`。token 是可用登录凭据，文档中统一脱敏；客户端必须在真实登录后保存服务端返回值，不能把 token 作为固定凭据。

6. 客户端伪代码

代码块

```
1  initLog();
2  initSingularAttribution();
3
4  SFUser cached = loadUser();
5  if (isLocallyValid(cached)) {
6      restoreCachedLoginState(cached);
7      refreshToken(cached); // Token 登录和刷
    新: /server/user/autoLoginreflushtoken
8      if (retryableBusinessFailure) {
9          deleteCachedLoginUser();
10         loginVisitor(storedAccountID); // /server/user/platformLogin
11     }
12 } else {
13     loginVisitor(storedAccountID); // /server/user/platformLogin,
    accountType=1
14 }
15 saveUser(uid, token, name, accountType, registerTime);
16 String userID = String.valueOf(uid);
17 thinkingData.login(userID);
18 Singular.setCustomUserId(userID);
19
20 SingularAttribution attribution = loadPersistedAttribution();
21 if (attribution != null) {
22     uploadUserAttribution(uid, attribution); // 每次登录成功后同步一次
23     // 仅在 code == 0 后标记本次同步成功；失败时保留数据并退避重试。
24 }
25 // 如果 Singular 回调晚于登录，回调取得归因后再次调用 uploadUserAttribution(uid,
    attribution)。
```

不要把 uid 当 token 使用；不要在客户端自行生成服务端 token；不要把游客 `accountID` 每次随机重置，否则会创建新游客账号导致产品数据丢失。

V1.1 广告收入采集接口

本文档用于客户端向 Report 服务批量上报广告收入、广告展示和广告点击事件。接口使用与 Server API 相同的域名，通过 `/report/` 路径区分服务，签名规则与 Server 服务一致。

客户端接入顺序：先按《客户端认证接口接入文档》完成登录并设置 `#account_id`，再将一条或多条事件批量发送到 Report API。客户端只需要配置一个 `SERVER_API_DOMAIN`。

1. 接口信息

代码块

```
1 POST https://{SERVER_API_DOMAIN}/report/data/report
2 Content-Type: application/x-www-form-urlencoded
```

其中 `{SERVER_API_DOMAIN}` 与《客户端认证接口接入文档》中使用的 Server API 域名相同。

当前接口通过表单字段 `data` 接收事件数据，不是直接接收 `application/json` 请求体。
`data` 的内容必须是 JSON 数组字符串，即使只有一条事件也必须使用数组。

1.1 请求头

携带以下 8 个请求头。公共请求头参考用户登录接口接入文档，由客户端网络层统一添加。

Header	是否必传	字段意思	本期 Android 示例/取值来源
appId	是	产品 App ID，与请求参数 appId 表示同一个产品	
appVersion	是	当前客户端版本	从 App 包版本读取，例如 1.0.0
sdkVersion	是	当前接入的 SDK版本	例如 1.5.0
deviceId	是	客户端稳定设备标识，用于设备关联和封禁校验	例如 android-device-uuid
platformId	是	发行平台 ID，不是 Android/iOS 标识	本次 Google Play Android 传 2
deviceTime	是	发起请求时的客户端设备时间，Unix 毫秒	例如 1787137000000
zoneOffset	是	设备当前时区相对 UTC 的偏移小时数	中国时区传 8
Accept-Language	是	客户端当前语言，用于服务端多语言消息和地区配置	例如 zh-CN、en-US

2. 表单参数

参数	类型	必填	说明
appId	integer	是	产品 App ID，用于查找客户端密钥
timestamp	string	是	请求发起时的 Unix 秒时间戳，注意不是毫秒
data	string	是	事件 JSON 数组字符串，至少 1 条；建议合并多条批量上报
sign	string	是	按 Server 服务相同规则生成的 32 位大写 MD5

请求约束：

- data 根节点必须是数组： `[{...}]`。
- 每批至少 1 条，文档不规定固定条数上限。建议客户端把一段时间内的多条事件合并上报，减少高频网络请求；同时应控制单次请求体大小，避免批次过大造成超时。

- 同一批事件应属于同一个 `appID`，每条事件的 `properties.#bundle_id` 必须对应该产品的包名。
- `data` 作为完整表单参数参与签名。签名后不得重新序列化或改变其空格、字段顺序、转义和字段值。
- 表单提交时由 HTTP 客户端执行 URL 编码，不要手工重复编码 `data`。

3. 支持的事件

本接口接收以下三类数数科技事件原始格式，后端按事件名称分别解析和保存：

事件	<code>#event_name</code>	用途
广告收入	<code>ad_revenue</code>	上报单次广告展示产生的收入、币种和广告来源
广告展示	<code>ad_impression</code>	上报一次广告展示
广告点击	<code>ad_click</code>	上报一次广告点击及点击持续时间等信息

客户端应保留数数科技事件的原始字段名，包括字段名中的 `#` 前缀。

4. 数数科技账号 ID

每条事件顶层必须补充 `#account_id`：

代码块

```
1  {
2    "#account_id": 1542198364433551363
3  }
```

`#account_id` 是数数科技的账号 ID，用于标识用户登录后的数据。本项目默认使用客户端认证接口登录成功后返回的 `data.uid`，不要使用游客 `accountID`、`#distinct_id`、`#uuid`、设备号或 Token 替代。

产品同时存在“账号”和“角色”两个维度时，可以按产品的数据分析口径使用更细粒度的角色 ID 作为 `#account_id`；没有角色维度时，使用认证服务返回的登录用户 `uid`。

数数科技客户端 SDK 的身份设置规则：

1. 用户注册、登录成功，或创建角色、进入服务器后，调用 SDK 的 `login` 设置账号 ID。
2. SDK 会保存该账号 ID，之后采集的事件会自动携带 `#account_id`。
3. 切换账号时再次调用 `login`，新值会覆盖旧值。
4. 用户退出登录时调用 `logout`，清除后续事件中的账号 ID。
5. 必须先设置账号 ID，再采集和批量上报本接口的三个广告事件。

Android 示例：

代码块

```
1 // thinkingData 为已初始化的数数科技 SDK 实例。
2 long uid = loginResult.getUid();
3 thinkingData.login(String.valueOf(uid));
4
5 // 用户退出当前账号时清除账号身份。
6 thinkingData.logout();
```

数数科技账号 ID 说明：<https://docs-v2.thinkingdata.cn/?version=v5.0&lan=zh-CN&code=user_identify&anchorId=>

5. 完整事件示例

以下示例保持客户端实际采集结构，并统一增加顶层 `#account_id`。示例账号 ID 仅用于说明，实际值必须取当前已登录用户身份。

5.1 ad_revenue

代码块

```
1  {
2    "#time": "2026-08-26 03:45:32.281",
3    "#distinct_id": "f5990108-a591-436b-a308-b7c8a3f793a9",
4    "#account_id": 1542198364433551363,
5    "#uuid": "51deb252-fced-44fa-8190-7dc38e0e8347",
6    "#event_id": 1542198364433551364,
7    "#type": "track",
8    "#event_name": "ad_revenue",
9    "properties": {
10     "#lib_version": "3.0.3.1",
11     "#os": "Android",
12     "#zone_offset": -4,
13     "#ram": "0.7/3.5",
14     "#data_source": "Native_SDK",
15     "#screen_height": 2340,
16     "#device_model": "SM-A156U",
17     "#system_language": "es",
18     "#network_type": "WIFI",
19     "ad_preload": false,
20     "network": "Google Adwords for Video - UAC",
21     "isDebug": false,
22     "campaign_name": "23771683571 - DRecover-Stan-Google-US-ES-3.0-0420 (585-036-5846)",
23     "#lib": "Android",
24     "#device_type": "Phone",
25     "ad_source": "AdMob",
26     "passthrough": "null",
27     "match_type": "null",
28     "currency": "USD",
29     "#disk": "23.1/45.8",
30     "value": 0.000153,
31     "campaign_id": "23771683571",
32     "ad_format": "native",
33     "#carrier": "",
34     "click_timestamp": "1779674123000000",
35     "#device_id": "dfaec1745686e4bc",
36     "#bundle_id": "com.toolrecovery.restorerecoverydata",
37     "#screen_width": 1080,
38     "traffic_source": "app_push",
39     "#install_time": "2026-05-24 21:56:30.860",
```

```

40     "#simulator": false,
41     "areakey": "coreFeaturesNativeAdv",
42     "ad_platform": "adMob",
43     "#fps": 60,
44     "ad_unit_name": "ca-app-pub-3615322193850391/3829965454",
45     "#manufacturer": "samsung",
46     "fromNature": false,
47     "#os_version": "16",
48     "#app_version": "1.0.20"
49 }
50 }

```

5.2 ad_impression

代码块

```

1  {
2      "#time": "2026-08-26 03:45:32.281",
3      "#distinct_id": "f5990108-a591-436b-a308-b7c8a3f793a9",
4      "#account_id": 1542198364433551363,
5      "#uuid": "8ff293e1-0365-4810-abd3-3f3c74ad78d9",
6      "#event_id": 1542198364433551363,
7      "#type": "track",
8      "#event_name": "ad_impression",
9      "properties": {
10         "#lib_version": "3.0.3.1",
11         "#os": "Android",
12         "#zone_offset": -4,
13         "#ram": "0.7/3.5",
14         "#data_source": "Native_SDK",
15         "#screen_height": 2340,
16         "#device_model": "SM-A156U",
17         "#system_language": "es",
18         "#network_type": "WIFI",
19         "ad_preload": false,
20         "network": "Google Adwords for Video - UAC",
21         "isDebug": false,
22         "campaign_name": "23771683571 - DRecover-Stan-Google-US-ES-3.0-0420 (585-036-5846)",
23         "#lib": "Android",
24         "#device_type": "Phone",
25         "ad_source": "AdMob",
26         "passthrough": "null",

```

```

27     "match_type": "null",
28     "currency": "USD",
29     "#disk": "23.1/45.8",
30     "value": 0.000153,
31     "campaign_id": "23771683571",
32     "ad_format": "native",
33     "#carrier": "",
34     "click_timestamp": "1779674123000000",
35     "#device_id": "dfaec1745686e4bc",
36     "#bundle_id": "com.toolrecovery.restorerecoverydata",
37     "#screen_width": 1080,
38     "traffic_source": "app_push",
39     "#install_time": "2026-05-24 21:56:30.860",
40     "#simulator": false,
41     "areakey": "coreFeaturesNativeAdv",
42     "ad_platform": "adMob",
43     "#fps": 60,
44     "ad_unit_name": "ca-app-pub-3615322193850391/3829965454",
45     "#manufacturer": "samsung",
46     "fromNature": false,
47     "#os_version": "16",
48     "#app_version": "1.0.20"
49 }
50 }

```

5.3 ad_click

代码块

```

1  {
2    "#time": "2026-08-26 03:43:44.613",
3    "#distinct_id": "d0079794-bb12-4e3d-91b7-c8a8624628f2",
4    "#account_id": 1542198364433551363,
5    "#uuid": "2587d9c8-6576-4823-9bc8-45a6b58925cf",
6    "#event_id": 1542197875763580928,
7    "#type": "track",
8    "#event_name": "ad_click",
9    "properties": {
10     "#lib_version": "3.0.3.1",
11     "#os": "Android",
12     "#zone_offset": -4,
13     "#ram": "1.7/5.3",
14     "#data_source": "Native_SDK",

```

```

15     "#screen_height": 2408,
16     "#device_model": "TMRV085G",
17     "#system_language": "en",
18     "#network_type": "5G",
19     "ad_preload": false,
20     "network": "Google Adwords for Video - UAC",
21     "isDebug": false,
22     "campaign_name": "23771683571 - DRecover-Stan-Google-US-ES-3.0-0420 (585-
036-5846)",
23     "#lib": "Android",
24     "#device_type": "Phone",
25     "ad_source": "AdMob Network",
26     "passthrough": "null",
27     "match_type": "null",
28     "#disk": "70.8/102.8",
29     "campaign_id": "23771683571",
30     "#carrier": "Metro by T-Mobile",
31     "click_timestamp": "1787730191000000",
32     "format": "open",
33     "#device_id": "0f8087179f46a412",
34     "#bundle_id": "com.toolrecovery.restorerecoverydata",
35     "#screen_width": 1080,
36     "traffic_source": "",
37     "#install_time": "2026-08-26 03:43:20.718",
38     "#simulator": false,
39     "areakey": "openPageAdv",
40     "ad_platform": "adMob",
41     "#fps": 60,
42     "ad_unit_name": "ca-app-pub-3615322193850391/7841254850",
43     "duration_time": 14356,
44     "#manufacturer": "Luxshare",
45     "fromNature": false,
46     "#os_version": "16",
47     "#app_version": "1.0.20"
48 }
49 }

```

6. 批量上报示例

一次请求可以把不同事件放在同一个数组中。下面省略 `properties` 的具体字段以展示数组结构；真实请求必须发送第 5 节所示的完整对象。

代码块

```
1  [
2    {
3      "#time": "2026-08-26 03:45:32.281",
4      "#distinct_id": "f5990108-a591-436b-a308-b7c8a3f793a9",
5      "#account_id": 1542198364433551363,
6      "#uuid": "51deb252-fced-44fa-8190-7dc38e0e8347",
7      "#event_id": 1542198364433551364,
8      "#type": "track",
9      "#event_name": "ad_revenue",
10     "properties": {}
11   },
12   {
13     "#time": "2026-08-26 03:45:32.281",
14     "#distinct_id": "f5990108-a591-436b-a308-b7c8a3f793a9",
15     "#account_id": 1542198364433551363,
16     "#uuid": "8ff293e1-0365-4810-abd3-3f3c74ad78d9",
17     "#event_id": 1542198364433551363,
18     "#type": "track",
19     "#event_name": "ad_impression",
20     "properties": {}
21   },
22   {
23     "#time": "2026-08-26 03:43:44.613",
24     "#distinct_id": "d0079794-bb12-4e3d-91b7-c8a8624628f2",
25     "#account_id": 1542198364433551363,
26     "#uuid": "2587d9c8-6576-4823-9bc8-45a6b58925cf",
27     "#event_id": 1542197875763580928,
28     "#type": "track",
29     "#event_name": "ad_click",
30     "properties": {}
31   }
32 ]
```

同一批可以继续追加多个完整事件对象。建议客户端按缓存条数、时间间隔或请求体大小触发批量发送，以减少高频网络请求。每个事件使用自己的 `#uuid` 和 `#event_id`，重试同一事件时必须复用原值。

7. 签名规则

签名算法与 Server 服务一致。参与签名的是表单参数 `appID`、`data` 和 `timestamp`，不包含 `sign`：

1. 删除 `sign`，忽略值为空的参数。
2. 参数名按 ASCII 升序排列。
3. 拼接为 `key=value&key2=value2&`。
4. 末尾追加 `secretKey={客户端密钥}`。
5. 对完整字符串计算 MD5，并转为 32 位大写十六进制字符串。

本接口排序后的签名原文为：

代码块

```
1  appID={APP_ID}&data={与实际发送内容完全一致的JSON数组字符串}&timestamp={UNIX_SECONDS}&secretKey={客户端密钥}
```

`{客户端密钥}` 从运营后台“基本信息配置”中取得。

JavaScript 示例：

代码块

```
1  function buildReportRequest(appID, clientKey, events) {
2    if (!Array.isArray(events) || events.length < 1) {
3      throw new Error("events must contain at least one item");
4    }
5
6    const timestamp = Math.floor(Date.now() / 1000).toString();
7    const data = JSON.stringify(events);
8    const raw =
9      `appID=${appID}&data=${data}&timestamp=${timestamp}&secretKey=${clientKey}`;
10   const sign = CryptoJS.MD5(raw).toString().toUpperCase();
11
12   return { appID: String(appID), timestamp, data, sign };
13 }
```

8. 请求示例

8.1 curl

代码块

```
1 curl --request POST "https://${SERVER_API_DOMAIN}/report/data/report" \  
2   --header "Content-Type: application/x-www-form-urlencoded" \  
3   --data-urlencode "appID=${REPORT_APP_ID}" \  
4   --data-urlencode "timestamp=${TIMESTAMP}" \  
5   --data-urlencode "data=${DATA_JSON_ARRAY}" \  
6   --data-urlencode "sign=${SIGN}"
```

8.2 Postman

Body 选择 `x-www-form-urlencoded` :

Key	Value
<code>appID</code>	<code>{{appID}}</code>
<code>timestamp</code>	<code>{{timestamp}}</code>
<code>data</code>	<code>{{reportData}}</code>
<code>sign</code>	<code>{{sign}}</code>

Pre-request Script:

代码块

```
1 const timestamp = Math.floor(Date.now() / 1000).toString();
```

```
2  const data = pm.environment.get("reportData");
3  const events = JSON.parse(data);
4
5  if (!Array.isArray(events) || events.length < 1) {
6    throw new Error("reportData must be a non-empty JSON array");
7  }
8
9  const appID = pm.environment.get("appID");
10 const clientKey = pm.environment.get("clientKey");
11 const raw =
    `appID=${appID}&data=${data}&timestamp=${timestamp}&secretKey=${clientKey}`;
12
13 pm.environment.set("timestamp", timestamp);
14 pm.environment.set("sign", CryptoJS.MD5(raw).toString().toUpperCase());
```

9. 响应与重试

接收成功示例：

代码块

```
1  {
2    "code": 0,
3    "msg": "operation success",
4    "serverTime": 1787729000200
5  }
```

签名失败示例：

代码块

```
1  {
2    "code": -1,
3    "msg": "sign not matched",
4    "serverTime": 1787729000200
5  }
```

处理要求：

- 必须以响应 `code == 0` 判断请求是否被接口接受，不能只判断 HTTP 200。
- `code == 0` 只表示整批数据已由接口接收并投入异步处理。
- 网络超时或非成功响应可以重试；重试时复用事件原有的 `#uuid` 和 `#event_id`。
- 建议使用有限次数重试、本地持久化队列和指数退避，避免无限高频重试。
- 不要把多个应用的数据混入同一批次。

10. 客户端发送前检查

1. `data` 根节点是非空数组，至少包含 1 条事件；建议多条合并上报以减少网络请求。
 2. 每条事件的 `#event_name` 是 `ad_revenue`、`ad_impression` 或 `ad_click`。
 3. 已登录用户的每条事件顶层包含 `#account_id`，值与当前登录身份一致。
 4. 每条事件的 `#uuid`、`#event_id` 非空，重试时不重新生成。
 5. 每条事件包含完整 `properties`，且 `properties.#bundle_id` 对应当前 `appID`。
 6. `ad_revenue` 的 `properties.value` 是合法数值，`properties.currency` 是三位货币代码。
 7. 对最终发送的 `data` 字符串计算签名，签名后不再重新序列化。
-

V1.0 包名校验+IP归属地(老版本)

本文档面向 Android SDK 调用方，当前包含 IP 归属地查询和 包名校验两个接口。

本版本接口由原 Go `PDFfox_server` 项目迁移至 Java 服务端，保留原接口路径、调用方式、原有请求参数和返回参数。两个接口新增可选的 `deviceID`（设备 ID）、`appID`（产品 ID），旧客户端不传也可正常调用，无需调整现有接入。

1. 通用约定

项目	约定
Base URL	由部署环境提供，例如 <code>https://api.example.com</code>
字符集	UTF-8
Content-Type	GET 接口无请求体；POST 接口使用 <code>application/json; charset=utf-8</code>
登录/Header	不要求登录态或 SDK 公共鉴权 Header
时间	<code>/parseToken</code> 的 <code>time</code> 格式为 <code>yyyy-MM-dd HH:mm:ss</code> ，使用服务端本地时间
HTTPS	生产环境必须使用 HTTPS

2. IP 归属地查询

2.1 接口信息

项目	内容
接口名称	IPInfo V2
方法	<code>GET</code>
路径	<code>/getIpInfoV2</code>
认证	无需登录，无需自定义 Header
请求参数	可选 Query 参数 <code>deviceId</code> 、 <code>appId</code> ，无请求体

接口根据当前 HTTP 请求识别客户端 IP，调用方不需要传递 IP 参数。

参数	类型	必填	说明
deviceID	string	否	客户端设备 ID；不传也可以，兼容线上旧客户端
appID	integer	否	产品 ID；不传也可以，兼容线上旧客户端

2.2 调用示例

代码块

```
1  curl --request GET "https://api.example.com/getIpInfoV2?deviceID=android-
    device-001&appID=20046" \
2  --header "Accept: application/json"
```

2.3 成功响应

HTTP 状态码： 200 OK

响应是裸 JSON 对象，不包含 data、success 或 code 外层包装。

代码块

```
1 {
2   "Ip": "8.8.8.8",
3   "IpLocation": "Mountain View",
4   "location_info": "{\"ip\":\"8.8.8.8\",\"city_name\":\"Mountain
  View\",\"asn\":15169,\"isp\":\"Google LLC\"}"
5 }
```

字段	类型	必返	说明
Ip	string	是	服务端识别出的客户端 IP，字段名大小写固定
IpLocation	string	是	城市名称，对应 IP2Location 的 city_name；没有城市时返回空字符串
location_info	string	是	IP2Location 返回的完整 JSON 字符串，不是嵌套 JSON 对象

location_info 内部常用字段包括 ip、city_name、longitude、latitude、asn、isp 等。调用方应先将该字符串解析为 JSON；当内部 ip 为空时，可使用外层 Ip 作为回退值。

2.4 异常响应

当前实现无法取得 IP、第三方请求失败或其他异常时返回 HTTP 500，响应使用兼容旧系统的 ObjectView 结构：

代码块

```
1 {
2   "code": 500,
3   "msg": "server error",
4   "serverTime": 1787653200000,
5   "data": null
```

`msg` 的实际国际化文本由服务端配置决定，调用方应以 HTTP 状态码判断请求失败，不应依赖具体文案。异常时不要把该响应当作成功的 IP 信息对象解析。

3. 包名校验

3.1 接口信息

项目	内容
接口名称	包名校验
方法	POST
路径	/parseToken
认证	无需登录，无需自定义 Header
Content-Type	application/json; charset=utf-8

3.2 请求参数

代码块

```
1  {
2    "Key": "TianWangGaiDiHu",
3    "PackageName": "com.datatool.photorecovery",
4    "token": "[PLAY_INTEGRITY_TOKEN]",
5    "deviceID": "android-device-001"
```

```
6   "appID": 20046
7 }
```

字段	类型	必填	说明
Key	string	是	SDK 服务密钥，当前值由服务端校验；字段名必须是大写 <code>K</code>
PackageName	string	是	Android 应用包名，字段名必须是大写 <code>P</code> 和 <code>N</code>
token	string	是	Android Standard Play Integrity SDK 生成的完整 token
deviceId	string	否	客户端设备 ID；可省略、传 <code>null</code> 或空字符串，兼容线上旧客户端
appID	integer	否	产品 ID；不传也可以，兼容线上旧客户端

`Key` 和 `PackageName` 的 JSON 大小写属于协议的一部分。不要改成 `key` 或 `packageName`。`PackageName` 必须是已配置的 Android 包名，不支持未知包名，也不应传 iOS bundle ID。服务端会先解码 token，再使用其中的包名信息完成校验。

3.3 调用示例

```
代码块
1  curl --request POST "https://api.example.com/parseToken" \
2    --header "Content-Type: application/json; charset=utf-8" \
3    --data
  '{"Key":"TianWangGaiDiHu","PackageName":"com.datatool.photorecovery","token":"[PLAY_INTEGRITY_TOKEN]","deviceId":"android-device-001","appID":20046}'
```

3.4 成功响应

业务成功和业务失败当前均返回 HTTP 200 OK，请同时读取根节点 status。

代码块

```
1  {
2    "status": 1,
3    "time": "2026-08-25 16:30:00",
4    "msg": "success",
5    "data": "{\"requestDetails\":
  {\"requestPackageName\": \"com.datatool.photorecovery\", \"timestampMillis\": \"..
  .\"}, \"appIntegrity\":
  {\"appRecognitionVerdict\": \"PLAY_RECOGNIZED\", \"packageName\": \"com.datatool.p
  hotorecovery\", \"versionCode\": \"63\"}, \"deviceIntegrity\":
  {...}, \"accountDetails\": {...}}\"
6  }
```

字段	类型	说明
status	integer	1 表示服务端成功；参数校验失败为 400；凭据、Google 调用或解析失败为 0
time	string	服务端生成时间，格式 yyyy-MM-dd HH:mm:ss
msg	string	success 或错误原因
data	string	包名校验成功时为 Google tokenPayloadExternal 的 JSON 字符串；失败时通常为 null

客户端应先读取 data 字符串，再进行一次 JSON 解析。不能按嵌套对象读取，也不能要求服务端把 verdict 转换成自定义字段。status=1 表示 token 已通过服务端包名校验。

3.5 失败响应

代码块

```
1  {
2    "status": 400,
3    "time": "2026-08-25 16:30:00",
4    "msg": "unsupported packageName",
5    "data": null
6  }
```

场景	status	msg
Key 错误或请求体为空	400	key error
PackageName 为空	400	packageName is empty
token 为空	400	token is empty
包名不存在或不匹配	400	unsupported packageName
凭据缺失、第三方校验或 token 解析失败	0	error

JSON 格式错误、缺少有效请求体等请求格式问题可能返回 4xx；调用方应对非 2xx 也做通用失败处理。

3.6 客户端处理建议

返回的 data 是 JSON 字符串，客户端需要先读取字符串，再解析为 JSON 对象。解析后可按需读取 requestDetails、appIntegrity、

deviceIntegrity、accountDetails 等 Google 返回字段，并根据业务需要处理 verdict。该接口的主要用途是校验 token 对应的 Android 包名是否与请求中的 packageName 一致。

版本记录

版本	变更内容	日期
V1.0	包名校验和IP归属地接口	2026-08-20
V1.1	广告收入采集接口	2026-09-09
V1.2	用户登录接口	2026-09-09
V1.0 更新	包名校验和IP归属地接口-参数增加设备ID、appID	2026-09-17
V1.3	配置查询接口	2026-09-21
V1.2 初始化接口更新	初始化接口（ /server/user/initLog ）新增 manufacturer、ram、disk字段	2026-09-29